Protobuf ★★★★

Что такое Protobuf

Protocol Buffers / Protobuf — это формат для описания и сериализации сообщений.

`Protobuf = .proto schema + generated classes + binary serialization.`

То есть Protobuf даёт:

1. .proto file
   Описание структуры сообщения.
2. Code generation
   Генерация классов для Java, Go, Python, Swift и других языков.
3. Binary serialization
   Превращение объекта в компактные binary bytes.

Protobuf часто используют с **gRPC**, но он не только для gRPC.

Его можно использовать и с:

```
Kafka
RabbitMQ / message brokers
internal service-to-service communication
mobile/backend communication
binary files
data pipelines
event-driven systems
```

Обычно в Java/Spring проекте:

Где обычно лежат `.proto` файлы

Обычно в Java/Spring проекте: `src/main/proto`

Пример структуры:

```
payment-service
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com/silo/payment/...
│   │   └── proto
│   │       └── payment_command.proto
```

```proto
syntax = "proto3";

package silo.payment;

option java_package = "com.silo.payment.proto";
option java_multiple_files = true;

message PaymentCommand {
  string command_id = 1;
  string booking_id = 2;
  string citizen_id = 3;
  int64 amount_chits = 4;
  PaymentCommandType type = 5;
}
enum PaymentCommandType {
  PAYMENT_COMMAND_TYPE_UNSPECIFIED = 0;
  CHARGE = 1;
  REFUND = 2;
}
```

Что значит `syntax = "proto3"`

Это версия языка `.proto`. Не версия твоего события.

Не версия Kafka topic-а. Не версия Schema Registry. Это значит:

`Этот файл написан по правилам Protobuf version 3 syntax.`

Важные особенности `proto3`:

```
В proto3 нет старого required как в proto2.
Поля не required на уровне Protobuf.
Если поле отсутствует, consumer получает default value.

string -> ""
number -> 0
boolean -> false
enum -> первое значение, обычно 0
message field -> can be not set, проверяется через hasXxx()
```

То есть если поле бизнес-обязательное, это надо проверять в коде:

```java
private void validate(PaymentCommand command) {
    if (command.getCommandId().isBlank()) {
        throw new InvalidPaymentCommandException("commandId is required");
    }
    if (command.getBookingId().isBlank()) {
        throw new InvalidPaymentCommandException("bookingId is required");
    }
    if (command.getCitizenId().isBlank()) {
        throw new InvalidPaymentCommandException("citizenId is required");
    }
    if (command.getAmountChits() <= 0) {
        throw new InvalidPaymentCommandException("amountChits must be positive");
    }
    if (command.getType() == PaymentCommandType.PAYMENT_COMMAND_TYPE_UNSPECIFIED) {
        throw new InvalidPaymentCommandException("type is required");
    }
}
```

Главная мысль:

```
Protobuf schema говорит: поле существует и имеет такой тип.
Business validation говорит: это поле обязательно и должно иметь допустимое значение.
```

Что значит `java_package`

`option java_package = "com.silo.payment.proto";`

Это говорит Protobuf compiler-у:

`Сгенерированные Java-классы положить в package: com.silo.payment.proto`

То есть `.proto` файл физически лежит тут: `src/main/proto/payment_command.proto`

А Java-класс будет в package: `package com.silo.payment.proto;`

Что значит `java_multiple_files = true`

Если `true`, Protobuf сгенерирует отдельные Java-файлы: `PaymentCommand.java`, `PaymentCommandType.java`

```
PaymentCommandProto.java
  ├── PaymentCommand
  └── PaymentCommandType
```

Если `false`, он может сгенерировать один большой outer class, внутри которого будут nested-классы.

`.proto` файл — это schema, не сообщение

Вот это schema:

```
message PaymentCommand {
  string command_id = 1;
  string booking_id = 2;
  string citizen_id = 3;
  int64 amount_chits = 4;
  PaymentCommandType type = 5;
}
```

Она описывает:

```
field 1 = command_id, string
field 2 = booking_id, string
field 3 = citizen_id, string
field 4 = amount_chits, int64
field 5 = type, PaymentCommandType
```

А реальное сообщение логически может быть таким:

```
command_id = "cmd-1"
booking_id = "booking-7"
citizen_id = "citizen-42"
amount_chits = 5000
type = REFUND
```

Но в Kafka оно не лежит как JSON.

В Kafka value будет: `protobuf binary bytes`

Как появляются Java-классы

Ты не пишешь руками:

```
public record PaymentCommand(
        String commandId,
        String bookingId,
        String citizenId,
        long amountChits,
        String type
) {}
```

При нормальном использовании Protobuf ты пишешь `.proto`.

Потом на build-е работает `protoc`.

```
payment_command.proto
        ↓ protoc / build plugin
generated Java classes
        ↓
PaymentCommand.java
PaymentCommandType.java
```

Использование generated class:

```java
PaymentCommand command = PaymentCommand.newBuilder()
        .setCommandId("cmd-1")
        .setBookingId("booking-7")
        .setCitizenId("citizen-42")
        .setAmountChits(5000L)
        .setType(PaymentCommandType.REFUND)
        .build();
```

Сериализация в bytes: `byte[] bytes = command.toByteArray();`

Десериализация из bytes: `PaymentCommand parsed = PaymentCommand.parseFrom(bytes);`

Модель:

```
.proto file
    ↓
generated Java class
    ↓
object
    ↓
toByteArray()
    ↓
protobuf binary bytes
    ↓
parseFrom(bytes)
    ↓
new Java object
```

Protobuf class — это не Spring bean

Generated `PaymentCommand` — это обычный Java data class.

Он не добавляется в Spring context.

Зачем field numbers: `= 1`, `= 2`, `= 3`

В Protobuf очень важны номера полей:

```proto
string command_id = 1;
string booking_id = 2;
string citizen_id = 3;
int64 amount_chits = 4;
PaymentCommandType type = 5;
```

В binary payload Protobuf обычно хранит не имена полей:

```
command_id
booking_id
amount_chits
```

А номера:

```
1
2
4
```

Упрощённо:

```
field 1 = "cmd-1"
field 2 = "booking-7"
field 3 = "citizen-42"
field 4 = 5000
field 5 = REFUND
```

JSON хранит имена полей в каждом сообщении:

```
{
  "commandId": "cmd-1",
  "bookingId": "booking-7",
  "amountChits": 5000
}
```

Protobuf вместо имён хранит номера и значения. Поэтому он обычно компактнее.

⚠ Главное правило Protobuf

`Never reuse field numbers.`

Плохо:

```proto
// old
int64 amount_chits = 4;

// bad new
string artist_id = 4;
```

Почему плохо? Старый consumer думает:

`field 4 = amount_chits`

А новый producer уже пишет:

`field 4 = artist_id` Смысл сломан.

Если поле удалили, номер лучше зарезервировать:

```proto
message PaymentCommand {
  reserved 4;
  reserved "amount_chits";

  string command_id = 1;
  string booking_id = 2;
  string citizen_id = 3;
  PaymentCommandType type = 5;
}
```

Это значит: `Номер 4 и имя amount_chits больше нельзя использовать для нового смысла.`

Enum в Protobuf

Лучше не делать так: `proto` `string type = 5;` Потому что тогда можно отправить что угодно:

```
"REFUND"
"refund"
"Refund"
"BANANA"
"R3FUND"
```

Лучше enum:

```proto
enum PaymentCommandType {
  PAYMENT_COMMAND_TYPE_UNSPECIFIED = 0;
  CHARGE = 1;
  REFUND = 2;
}
```

И поле:

```proto
PaymentCommandType type = 5;
```

Почему первое значение `0` ?

В `proto3` default enum value — первое значение.

Поэтому хорошая практика:

```proto
PAYMENT_COMMAND_TYPE_UNSPECIFIED = 0;
```

Если producer не задал type, consumer увидит: `PAYMENT_COMMAND_TYPE_UNSPECIFIED`

и бизнес-валидация может сказать: `type is required`

Message field / nested message

Можно делать вложенные структуры.

```proto
message PaymentCommand {
  string command_id = 1;
  string booking_id = 2;
  string citizen_id = 3;
  ChitTransfer chit_transfer = 4;
  PaymentCommandType type = 5;
}
message ChitTransfer {
  int64 amount = 1;
  string unit = 2;
}
```

Вот это: `ChitTransfer chit_transfer = 4;`

это `message field`.

Аналогия в Java:

```
public record PaymentCommand(
        String commandId,
        String bookingId,
        String citizenId,
        ChitTransfer chitTransfer,
        PaymentCommandType type
) {}
public record ChitTransfer(
        long amount,
        String unit
) {}
```

В generated Protobuf Java-коде обычно проверяют presence так:

```
if (command.hasChitTransfer()) {
    ChitTransfer transfer = command.getChitTransfer();
}
```

Protobuf без Schema Registry

Можно использовать Protobuf без Schema Registry. Тогда схема такая:

```
Producer has .proto/generated class
        ↓
creates PaymentCommand object
        ↓
command.toByteArray()
        ↓
Kafka value = protobuf bytes
        ↓
Consumer has compatible .proto/generated class
        ↓
PaymentCommand.parseFrom(bytes)
```

В Kafka летит: `protobuf binary payload`

Без schema id. Без полного `.proto` файла.

Consumer должен заранее знать, каким классом читать bytes.

То есть на consumer side должен быть совместимый generated class.

Protobuf with Confluent Schema Registry

В Spring Kafka producer-е связь со Schema Registry задаётся настройками:

```
spring.kafka.producer.value-serializer=io.confluent.kafka.serializers.protobuf.KafkaProtobufSerializer
spring.kafka.properties.schema.registry.url=http://schema-registry:8081
```

Важно:

```
KafkaTemplate не знает про Schema Registry.
KafkaProtobufSerializer знает про Schema Registry.
schema.registry.url говорит serializer-у,
куда идти за schema id.
```

Цепочка:

```
kafkaTemplate.send(...)
        ↓
KafkaProducer
        ↓
value serializer
        ↓
KafkaProtobufSerializer
        ↓
Schema Registry lookup/register
        ↓
       byte[]
        ↓
Kafka broker
```

Что хранится в generated class

После build-а generated class содержит две вещи:

```
1. Код для хранения значений.
2. Static descriptor/schema metadata.
```

Упрощённо:

```
PaymentCommand.class contains descriptor:
message PaymentCommand
field 1 = command_id, string
field 2 = booking_id, string
field 3 = citizen_id, string
field 4 = amount_chits, int64
field 5 = type, enum
```

Это не Java object header. Это не Klass Pointer.

Это не обычная reflection над POJO. Это metadata, которую `protoc` сгенерировал из `.proto` файла.

Что отправляется в Schema Registry

Он берёт descriptor из generated class и отправляет schema representation в Schema Registry:

```
"message PaymentCommand {
   string command_id = 1;
   string booking_id = 2;
   string citizen_id = 3;
   int64 amount_chits = 4;
   PaymentCommandType type = 5;
}"
```

Schema Registry отвечает: `schema id = 102`

Что отправляется в Kafka

В Kafka летит примерно:

```
schema id = 102
+
protobuf payload:
field 1 = "cmd-1"
field 2 = "booking-7"
field 3 = "citizen-42"
field 4 = 5000
field 5 = REFUND
```

Упрощённо: `Kafka value = schema id + protobuf binary payload`

Более точная внутренняя структура Confluent не важна для собеса; главное держать модель:

```
Full schema goes to Schema Registry.
Schema id goes to Kafka with each message.
Full .proto does not go to Kafka with each message.
```

Producer cache

Serializer не обязан каждый раз ходить в Schema Registry.

Модель:

```
First message with PaymentCommand schema:
serializer reads descriptor
        ↓
lookup/register schema in Schema Registry
        ↓
gets schema id = 102
        ↓
caches schema -> id

Next messages with same schema:
serializer uses cached id 102
        ↓
sends schema id + payload to Kafka
```

То есть:

`Schema may be sent to Registry during first use / cache miss / app restart.`

Consumer with Schema Registry

Consumer properties:

```
spring.kafka.consumer.value-deserializer=io.confluent.kafka.serializers.protobuf.KafkaProtobufDeserializer
spring.kafka.properties.schema.registry.url=http://schema-registry:8081
spring.kafka.consumer.properties.specific.protobuf.value.type=com.silo.payment.proto.PaymentCommand
```

```
@KafkaListener(topics = "payment-commands", groupId = "payment-executor")
public void handle(PaymentCommand command) {
    // normal business logic
}
```

Почему producer не указывает `specific.protobuf.value.type`, а consumer указывает?

Producer уже держит объект: `PaymentCommand command = ...` Serializer видит runtime type: `object type = PaymentCommand`

Consumer получает только bytes: `schema id + protobuf payload` И ему надо понять: `Во что превратить эти bytes?`

Очень важно: `Schema Registry does not update running Java classes.`

Если consumer настроен так: `specific.protobuf.value.type=com.silo.payment.proto.PaymentCommand`

то listener получает именно этот generated Java class.

Так обычный Spring consumer не работает.

Для нормального business consumer-а новый Java class появляется только после:

```
изменили .proto
        ↓
build/protoc сгенерировал новый class
        ↓
собрали jar/docker image
        ↓
задеплоили consumer v2
```

Consumer не делает:

```
schema id = 102
        ↓
скачал .proto
        ↓
скомпилировал новый Java class
        ↓
подгрузил его в JVM
```

Consumer cache

Consumer тоже не обязан каждый раз ходить в Schema Registry.

Модель:

```
Kafka record:
schema id = 102
        ↓
consumer cache empty
        ↓
deserializer fetches schema 102 from Schema Registry
        ↓
caches id -> schema/descriptor
        ↓
deserializes payload
```

```
Next record:
schema id = 102
        ↓
schema already in cache
        ↓
no need to fetch again
```

Да, **в твоём конкретном варианте**: `specific.protobuf.value.type=com.silo.payment.proto.PaymentCommand`

и `public void handle(PaymentCommand command)` **Registry не нужен, чтобы выбрать Java-класс.** Класс уже указан.

Но если ты используешь именно:

```
KafkaProtobufDeserializer
schema.registry.url=http://schema-registry:8081
```

то deserializer работает по Confluent-формату: `schema id + protobuf payload`

И при первом встреченном `schema id` / cache miss он **может сходить в Schema Registry**, чтобы получить writer schema metadata, связанную с этим id.

Самый честный ответ:

```
В typed consumer-е этот lookup часто не нужен бизнес-логике.
Он нужен Confluent deserializer-у как части его Schema Registry protocol:
record содержит schema id, deserializer связывает этот id с registered writer schema.
Но listener всё равно получит PaymentCommand,
потому что Java type задан через specific.protobuf.value.type.
```

То есть да, твоя мысль правильная: `Для простого typed consumer-а это выглядит почти избыточно.`

DynamicMessage

```
DynamicMessage happens when the consumer does not use a specific generated class.
```

Например:

```
spring.kafka.consumer.value-deserializer=io.confluent.kafka.serializers.protobuf.KafkaProtobufDeserializer
spring.kafka.properties.schema.registry.url=http://schema-registry:8081
# no specific.protobuf.value.type
```

Тогда listener может быть generic:

```
@KafkaListener(topics = "payment-commands", groupId = "debug-reader")
public void handle(DynamicMessage message) {
    System.out.println(message);
}
```

Это нужно для:

```
debug tools
audit consumers
generic data platforms
connectors
schema browsers
```

Тогда schema id реально помогает:

```
schema id
   ↓
Schema Registry
   ↓
schema descriptor
   ↓
DynamicMessage
```

Но это не обычный удобный business-код.
В обычном Spring business service чаще используют specific generated class.

Старый consumer читает новое поле

v1:

```
proto
message PaymentCommand {
  string command_id = 1;
  string booking_id = 2;
  string citizen_id = 3;
  int64 amount_chits = 4;
  PaymentCommandType type = 5;
}
```

v2 добавила поле:

```
message PaymentCommand {
  string command_id = 1;
  string booking_id = 2;
  string citizen_id = 3;
  int64 amount_chits = 4;
  PaymentCommandType type = 5;
  string tattoo_zone = 6;
}
```

Producer v2 отправляет:

```
field 1 = cmd-1
field 2 = booking-7
field 3 = citizen-42
field 4 = 5000
field 5 = REFUND
field 6 = FOREARM
```

Old consumer v1 знает только fields `1..5`. Он видит field `6` и обычно игнорирует его.

Listener получает старый объект:

```
@KafkaListener(topics = "payment-commands", groupId = "payment-executor")
public void handle(PaymentCommand command) {
    System.out.println(command.getCommandId());
    System.out.println(command.getBookingId());
    System.out.println(command.getCitizenId());
    System.out.println(command.getAmountChits());
    System.out.println(command.getType());
    // getTattooZone() здесь нет,
    // потому что consumer v1 generated class не знает field 6.
}
```

То есть:

```
Old consumer can ignore new fields.

But old consumer cannot use new fields
until it is rebuilt and redeployed with the new .proto.
```

Compatibility modes

Schema Registry может проверять, совместима ли новая версия schema со старой. Основные режимы:

Mode	Смысл
`BACKWARD`	новый consumer может читать старые messages
`FORWARD`	старый consumer может читать новые messages
`FULL`	и backward, и forward
`NONE`	compatibility не проверяется

Для собеса чаще всего достаточно понимать `BACKWARD`.

"А если registry сказал forward-compatible, но parser не умеет игнорить?"

Да, тогда runtime может упасть. Правильная модель:

```
Schema Registry compatibility
= gate before producing.

Deserializer/parser behavior
= real runtime decoding.

Application code
= business processing.
```

Все три слоя должны быть нормальными. Schema Registry не маг.
Если теоретически compatibility policy пропустила изменение, но конкретный parser/deserializer или application code не может это обработать, будет ошибка.
Для Protobuf добавление нового поля с новым номером обычно нормально, потому что Protobuf parser умеет работать с unknown fields.

Safe Protobuf schema evolution

Обычно безопасно:

```
✅ add new field with new number
✅ keep old field numbers forever
✅ reserve deleted field numbers
✅ make new consumers handle missing/default values
✅ old consumers can usually ignore unknown new fields
```

Опасно:

```
❌ reuse field numbers
❌ change field meaning
❌ change field type in place
❌ remove fields too early
❌ remove enum values that may still appear
❌ add enum values without checking old consumer logic
```

Что такое wire-incompatible type

`wire` = binary format, который реально лежит в bytes.

Protobuf хранит не просто значение, а примерно: `field number + wire type + value`

Например:

```
proto
int64 amount_chits = 4;
```

`int64` кодируется одним способом.

А: `string amount_chits = 4;` `string` кодируется другим способом.

Если ты оставил тот же номер `4`, но поменял тип:

```
// old
int64 amount_chits = 4;

// bad new
string amount_chits = 4;
```

старый consumer ожидает: `field 4 = число` а новый producer пишет: `field 4 = строка`

Старый consumer может не прочитать это поле, проигнорировать его или получить default value.

То есть данные потеряны или смысл сломан. Поэтому правило простое:

```
Do not change Protobuf field type in place.
Add a new field with a new number.
```

Нормально:

```
// old
int64 amount_chits = 4;

// new
int64 amount_chits = 4;  // keep old for compatibility
string amount_text = 6;  // new field
```

Example: changing field meaning

Плохо:

```
proto
int64 amount_chits = 4;
```

Раньше: `amount_chits = сколько chits вернуть citizen-y`

Потом команда решила: `amount_chits = сколько chits citizen потратил за месяц`

Тип тот же. Номер тот же. Schema может выглядеть совместимой.

Но бизнес-смысл сломан. Правило:

```
Do not change field meaning in place.
Add a new field or create a new event/contract.
```

Broker-side schema ID validation

```
Broker-side schema ID validation protects Kafka topics from producers
that bypass Schema Registry and send records with fake or unregistered schema ids.
```

Bad/raw producer:

```
manually created byte[]
        ↓
fake schema id
        ↓
Kafka
```

Broker-side validation может сказать:

```
schema id 999999 is not registered
        ↓
reject record
```

Normal producer:

```
PaymentCommand object
        ↓
KafkaProtobufSerializer
        ↓
real schema id from Schema Registry
        ↓
Kafka
```

Это edge-case защита. В обычном приложении Confluent serializer сам получает id из Schema Registry.

✅ Преимущества Protobuf

1. Compact binary format
   В payload не нужно хранить имена полей, используются field numbers.
2. Faster serialization/deserialization
   Обычно быстрее и компактнее JSON.
3. Strong schema
   .proto явно описывает поля, типы, enum values.
4. Code generation
   Можно генерировать классы для Java, Go, Python, Swift и других языков.
5. Cross-language communication
   Один .proto contract могут использовать сервисы на разных языках.
6. Good for gRPC
   Protobuf — стандартный формат для gRPC.
7. Good for Kafka with Schema Registry
   Можно хранить версии schemas и проверять compatibility.

❌ Слабые стороны Protobuf

1. Not human-readable by default
   Binary payload нельзя просто открыть глазами как JSON.
2. Requires build tooling
   Нужен protoc / build plugin / generated classes.
3. Requires discipline with field numbers
   Нельзя переиспользовать номера.
4. Defaults can hide missing fields
   Например missing string becomes "",
   missing number becomes 0.
5. Less convenient for public/browser APIs
   Для REST/debug/curl/Postman JSON часто проще.
6. Schema changes need careful rollout
   Особенно если несколько producers/consumers живут одновременно.

⚖️ JSON vs Protobuf

JSON:

- text-based;
- readable by humans;
- field names are stored in every message;
- easy to debug with curl/Postman;
- schema optional, can work by convention.

Protobuf:

- binary;
- not directly human-readable;
- field numbers are stored instead of names;
- requires .proto schema;
- generates classes;
- compact and strict;
- good for internal service communication.

Коротко:

JSON can work by convention.
Protobuf works from a schema file.

🧠 One-line memory hooks

Protobuf = .proto schema → generated class → binary bytes.

Kafka stores bytes, not Java objects and not .proto files.

Field numbers are the real identity of Protobuf fields.
Never reuse them.